The Journal of Open Source Software

Pyxations: organizing, parsing, and analysing eye-movement data using Python

Gonzalo Ruarte [1], **Gustavo Juantorena** [1], **Joaquín González** [1,3], **and Juan Esteban Kamienkowski** [1,2]

1 Laboratorio de Inteligencia Artificial Aplicada, Instituto de Ciencias de la Computación, Facultad de Ciencias Exactas y Naturales, Universidad de Buenos Aires - CONICET, Argentina **2** Maestría de Explotación de Datos y Descubrimiento del Conocimiento, FCEyN-FI, UBA, Argentina **3** Departamento de Física, Facultad de Ciencias Exactas y Naturales, Universidad de Buenos Aires

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Britta Westner

Reviewers:

- @jgeller112
- @scott-huberty

Submitted: 26 December 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0)

Summary

Pyxations is a Python-based toolbox designed to unify the organization, parsing, and analysis of eye-movement data. It supports multiple common recording formats and detection algorithms, and offers integrated tools for preprocessing, visualization, and downstream analysis. Pyxations converts sample-level recordings from its supported eye trackers to the eye-tracking physiological format defined by BIDS and writes processed samples and detected eye movements to a linked, validator-tested BIDS Derivatives dataset using the general derivatives conventions and physiological recording file types. By standardizing these steps, Pyxations facilitates reproducibility and makes it easier to compare results across tasks, devices, and studies.

Statement of Need

Looking at someone's eyes, searching for keys, or reading are active processes in which eye movements play a crucial role (Holmqvist et al., 2011; Land & Tatler, 2009; Rayner, 1998; Wade & Tatler, 2005). Eye-tracking recordings can describe not only fixations and saccades, but also smooth pursuit, microsaccades, vergence, blinks, and changes in pupil size. Blink timing and pupil dynamics can be scientifically informative in their own right and should therefore be retained even when an analysis primarily concerns gaze position (Zhang & Jonides, 2026). Common remote laboratory eye-trackers use one or more cameras, typically with infrared illumination, to image the pupil and corneal reflection; pupil–corneal-reflection methods then estimate where the eyes are directed. Webcam-based systems generally operate without controlled infrared illumination and instead estimate gaze from eye and face images using computer-vision or machine-learning techniques (Juantorena et al., 2023). Continuous gaze and pupil signals may subsequently be classified into eye-movement events. Commercial systems such as EyeLink provide vendor-reported event classifications through a built-in parser (SR Research Ltd., 2021), whereas open algorithms include the methods of Engbert and Mergenthaler (2006) and REMoDNaV (Dar et al., 2021).

The Brain Imaging Data Structure (BIDS) and the Eye-Tracking-BIDS extension now specify how sample-level gaze position, pupil recordings, and their metadata should be represented (Gorgolewski et al., 2016; Szinte et al., 2026). However, converting heterogeneous vendor recordings into this representation and continuing from standardized samples to reproducible preprocessing, event detection, and analysis still requires substantial format-specific orchestration. Differences among experimental paradigms, devices, calibration procedures, and event-detection algorithms make it difficult to move from raw recordings to comparable results. Pyxations addresses this implementation gap while retaining the original vendor files for provenance.

DRAFT

In recent years, several open-source toolboxes have been developed to support eye-tracking data processing, including PyMovements (Krakowczyk & others, 2023), PyTrack (Ghose et al., 2020), SPEED (Lozzi et al., 2025), and PupEyes (Zhang & Jonides, 2026). Each offers valuable functionality within the growing ecosystem of open tools for gaze analysis. Below we summarize their key features and how Pyxations extends or complements them.

PyMovements provides a modular Python interface for loading datasets, preprocessing gaze data, detecting oculomotor events, computing data-quality measures, and rendering plots. Its dataset definitions and configurable paths support both public and local datasets. Pyxations has a different emphasis: it converts supported vendor exports into paired raw and derivative BIDS datasets and carries that organization through preprocessing and hierarchical analysis.

PyTrack is an end-to-end toolkit featuring fixation, saccade, and microsaccade extraction, estimation of multiple metrics (including pupil and blinks), visualizations, and statistical analysis. Its graphical interface makes these workflows accessible to non-programmers. Pyxations complements that approach with explicit multi-vendor conversion, BIDS-organized raw and derivative datasets, and machine-readable preprocessing provenance.

SPEED (LabSoC Standardized Processing and Extraction of Eye-tracking Data) also focuses on lowering the entry barrier through a graphical feature-extraction workflow for Pupil Labs Neon data. Pyxations instead targets scriptable, multi-vendor conversion and analysis within a BIDS-organized workflow.

PupEyes provides a dedicated pupil-preprocessing and quality-control workflow, including deblinking, artifact rejection, smoothing, interpolation, baseline correction, and interactive diagnostic visualizations (Zhang & Jonides, 2026). Pyxations does not duplicate this specialized pupillometry pipeline. Instead, it preserves pupil measurements and blink events when they are available, exposes them through `pupil_samples()` and `blinks()` at each level of its experiment hierarchy, and provides standardized tabular data that can be converted for use by downstream analysis packages.

Therefore, Pyxations was designed as a reproducible and extensible framework that unifies these complementary strengths while addressing the limitations of these other software packages.

Research Impact Statement

Pyxations has already been deployed to standardize workflows in active research environments. It was utilized to process and analyze webcam-based eye-tracking data for a study on validation of an online antisaccade paradigm (Juantorena et al., 2023). Moreover, source code was also utilized for EyeLink data in a study on hybrid search strategies, which combined Bayesian and neural network models (Ruarte et al., 2025).

The library's versatility has been further evaluated through the harmonization of diverse datasets. It has been used to process data from GazePoint 3 trackers, in-house web-based eye-trackers, and driving simulation experiments (courtesy of collaborators mentioned in the acknowledgements), demonstrating its capacity to handle real-world heterogeneity beyond standard laboratory tasks. Furthermore, we are currently collaborating with three additional research groups to integrate Pyxations into their data analysis pipelines, providing additional opportunities to evaluate its portability and reproducibility.

Software Design

Pyxations is designed as a reproducible and extensible framework that unifies the complementary strengths of previous developments while addressing their limitations. Its core contributions can be summarized (Fig. 1) as follows:

DRAFT

Standardized dataset organization. Pyxations writes sample-level eye-tracking data and metadata according to BIDS (Gorgolewski et al., 2016), including separate physiological recordings and corresponding tracker-event files for each eye, while retaining original vendor files under `sourcedata`. Processed sample streams and detected fixations, saccades, blinks, and messages are stored as compressed tabular files with JSON sidecars in a linked dataset that follows the general BIDS Derivatives conventions. BIDS does not yet define a domain-specific derivative schema for detected eye movements, so Pyxations documents its additional columns and processing provenance in those sidecars rather than claiming a separately standardized eye-movement derivative format. At the time of this revision, the generated raw and derivative datasets validate without errors against BIDS 1.11.1 using the official BIDS Validator 3.0.1 in continuous integration. This facilitates transparent sharing, version control, and collaborative reuse.

Cross-format parsing and harmonization. It supports EyeLink .edf/.asc, Tobii text exports, GazePoint CSV exports, and webcam recordings produced through WebGazer through a unified conversion API, reducing friction when combining data from different acquisition systems. Associated behavioral CSV/TSV files and PsychoPy trial logs can be normalized into BIDS events tables; PsychoPy-local timestamps are retained without assuming synchronization to the eye-tracker clock. Pupil measurements and blink events are retained when they are present in the source recording rather than being discarded during harmonization.

Calibration- and validation-aware preprocessing. Pyxations retains calibration reports and parses EyeLink VALIDATION blocks, extracting average and maximum error and offset values. These values can be inspected and used to exclude missing or poorly calibrated trials.

Flexible trial segmentation. The framework accommodates multiple trialing paradigms: explicit start and end timestamps, event-based markers, or fixed-duration trials. All of them with overlap controls and regular-expression message matching. This flexibility enables consistent parsing of diverse experimental logics without manual preprocessing.

Declarative, provenance-aware workflow. Configured trial-segmentation operations and their parameters are logged in machine-readable JSON recipes and provenance sidecars alongside the detection algorithm. This makes the transformations used to create each derivative dataset explicit and repeatable.

Scalability and performance. Pyxations uses the Polars data engine for in-memory preprocessing and analysis transformations (Vink, 2022) and can process independent recordings in parallel. These choices are intended to support multi-subject, multi-session analyses without making an unbenchmarked performance claim against other dataframe libraries.

Visualization, statistics, and inspection tools. In addition to standard gaze plots, Pyxations includes optional animated gaze visualizations, hierarchical data analysis (experiment, subject, session, trial), calibration-error visualization across trials, and task-specific visualization utilities tailored to paradigms such as visual search. Fixations, saccades, blinks, gaze samples, and pupil-containing samples can be retrieved at every level of the hierarchy. Optional MultiMatch metrics (Dewhurst et al., 2012) are available at the trial level to compare similarity between scanpaths.

Interoperability. Pyxations exposes standardized Polars tables for gaze samples, fixations, saccades, blinks, and pupil measurements, allowing downstream conversion or use by other Python analysis tools. It complements specialized packages such as PyMovements (Krakowczyk & others, 2023), PyTrack (Ghose et al., 2020), and PupEyes (Zhang & Jonides, 2026).

DRAFT

Figure 1

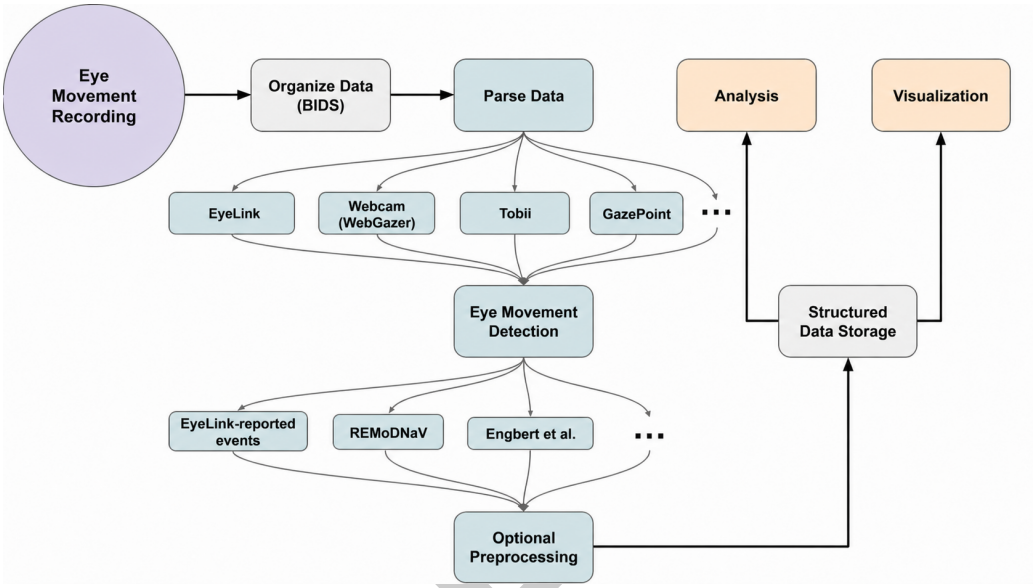


Figure 1: Preprocessing and analysis workflow showing parsing, event detection, preprocessing, and analysis stages.

In summary, Pyxations extends current open-source eye-tracking software by offering a scalable, provenance-aware, and calibration-informed framework that unifies parsing, preprocessing, and analysis within a standardized data structure. Its detector selection and experiment hierarchy are designed to support further input formats and task-specific analyses without changing the canonical BIDS storage layer.

AI Usage Disclosure

We utilized generative AI tools (primarily ChatGPT-4o and Gemini 2.5 Flash) to a limited extent, specifically for generating individual methods based on established architectural decisions, creating unit tests, and drafting documentation strings. We did not employ autonomous AI agents or large-scale automated coding pipelines. Regarding the manuscript, these tools were used solely for linguistic refinement, such as typo spotting, grammar correction, and minor stylistic improvements. The core scientific content, figures, and bibliography were not generated by AI.

Acknowledgements

This project was supported by CONICET and UBA. We thank Pablo Laciana for his contribution to the code and Damián Care, Fermín Travi and Bruno Bianchi for their expert insights and constructive discussions during the preparation of this work. We thank Stephanie Muller and Enzo Tagliazucchi for sharing the GazePoint 3 data. Finally, We express our gratitude to Matias Ison for sharing the Driving data.

References

Dar, H., Wagner, G., & Einhäuser, W. (2021). REMoDNaV: Robust eye-movement classification for dynamic stimulation. *Behavior Research Methods*, *53*, 1338–1360. https://doi.org/10.

DRAFT

3758/s13428-020-01428-x

Dewhurst, R., Nyström, M., Jarodzka, H., Foulsham, T., Johansson, R., & Holmqvist, K. (2012). It depends on how you look at it: Scanpath comparison in multiple dimensions with MultiMatch, a vector-based approach. *Behavior Research Methods*, *44*(4), 1079–1100. https://doi.org/10.3758/s13428-012-0212-2

Engbert, R., & Mergenthaler, K. (2006). Microsaccades are triggered by low retinal image slip. *Proceedings of the National Academy of Sciences*, *103*(18), 7192–7197. https://doi.org/10.1073/pnas.0509557103

Ghose, U., Srinivasan, A. A., Boyce, W. P., Xu, H., & Chng, E. S. (2020). PyTrack: An end-to-end analysis toolkit for eye tracking. *Behavior Research Methods*, *52*(6), 2588–2603. https://doi.org/10.3758/s13428-020-01392-6

Gorgolewski, K. J., Auer, T., Calhoun, V. D., Craddock, R. C., Das, S., Duff, E., & others. (2016). The brain imaging data structure, a format for organizing and describing outputs of neuroimaging experiments. *Scientific Data*, *3*, 160044. https://doi.org/10.1038/sdata.2016.44

Holmqvist, K., Nyström, M., Andersson, R., Dewhurst, R., Jarodzka, H., & Weijer, J. van de. (2011). *Eye tracking: A comprehensive guide to methods and measures*. Oxford University Press. https://doi.org/10.1080/17470218.2015.1098709

Juantorena, G. E., Figari, F., Petroni, A., & Kamienkowski, J. E. (2023). Web-based eye-tracking for remote cognitive assessments: The anti-saccade task as a case study. *bioRxiv*. https://doi.org/10.1101/2023.07.11.548447

Krakowczyk, M., & others. (2023). PyMovements: An open-source python package for eye movement analysis. *Journal of Open Source Software*, *8*(86), 5235. https://doi.org/10.1145/3588015.3590134

Land, M. F., & Tatler, B. W. (2009). *Looking and acting: Vision and eye movements in natural behaviour*. Oxford University Press. https://doi.org/10.1093/acprof:oso/9780198570943.001.0001

Lozzi, D., Di Pompeo, I., Marcaccio, M., Ademaj, M., Migliore, S., & Curcio, G. (2025). SPEED: A graphical user interface software for processing eye tracking data. *NeuroSci*, *6*(2). https://doi.org/10.3390/neurosci6020035

Rayner, K. (1998). Eye movements in reading and information processing: 20 years of research. *Psychological Bulletin*, *124*(3), 372–422. https://doi.org/10.1037/0033-2909.124.3.372

Ruarte, G., Bujia, G., Care, D., Ison, M. J., & Kamienkowski, J. E. (2025). Integrating bayesian and neural networks models for eye movement prediction in hybrid search. *Scientific Reports*, *15*(1), 16482. https://doi.org/10.1038/s41598-025-00272-3

SR Research Ltd. (2021). *Eyelink 1000 plus user manual*.

Szinte, M., Bach, D. R., Draschkow, D., Esteban, O., Gagl, B., Gau, R., Gregorova, K., Halchenko, Y. O., Huberty, S., Kling, S. M., Kulkarni, S., & others. (2026). Eye-tracking-BIDS: The brain imaging data structure extended to gaze position and pupil data. *bioRxiv*. https://doi.org/10.64898/2026.02.03.703514

Vink, R. (2022). *Polars: DataFrame library for rust and python*.

Wade, N. J., & Tatler, B. W. (2005). *The moving tablet of the eye: The origins of modern eye movement research*. Oxford University Press. https://doi.org/10.1093/acprof:oso/9780198566175.001.0001

Zhang, H., & Jonides, J. (2026). PupEyes: An interactive python library for eye movement data processing. *Behavior Research Methods*, *58*(1), 29. https://doi.org/10.3758/s13428-025-02830-z

DRAFT